SPM Module 1: Vectorization of the Partition Mapping Kernel

Gabriele Marsili

1 Design Choices

Problem. Given N keys (uint64_t) and P partitions (power of two), compute part_id$[i] = h(\text{keys}[i]) \in [0, P)$ for each key.

1.1 Hash Function: XOR-fold + Fibonacci mul32

The mapping function is:

$$h(k) = \big((k_{\text{lo}} \oplus k_{\text{hi}}) \cdot A_{32}\big) \gg (32 - \log_2 P) \qquad (1)$$

where k_{lo}, k_{hi} are the lower and upper 32 bits of the 64-bit key, and $A_{32} = \texttt{0x9E3779B9} = \lfloor 2^{32}/\varphi \rfloor$ is the Fibonacci constant (Knuth, TAOCP Vol. 3).

Why this function. The choice is driven by three considerations:

1. **SIMD-native**: the core operation is a 32-bit multiply, which maps to `vpmulld`—a native AVX2 instruction processing 8 elements at once. A 64-bit hash would need 3× `vpmuludq` decomposition, since AVX2 lacks `mullo_epi64`.
2. **Entropy preservation**: $k_{\text{lo}} \oplus k_{\text{hi}}$ combines both key halves; the Fibonacci constant provides quasi-universal hashing (collision $\leq 2/P$).
3. **Minimal computation**: XOR + mul32 + shift per key—no division, no modulo, no branch.

Distribution quality. Measured on 10^8 keys with $P = 256$: max(count)/expected = 1.005, confirming near-uniform distribution.

1.2 Data Layout

Keys and partition IDs are stored in separate contiguous arrays (`uint64_t[]` and `uint32_t[]`), accessed with unit stride. This SoA-like layout is optimal for vectorization. Memory is aligned to 32 bytes via `posix_memalign` for efficient AVX2 loads.

2 Auto-vectorization Evidence

The kernel is compiled from the same source into two binaries:

- **Baseline**: `-O3 -march=native -fno-tree-vectorize`
- **Autovec**: `-O3 -march=native -mavx2 -ftree-vectorize`

GCC 12.2 reports successful vectorization:

```
src/plain.cpp:33:26: optimized: loop vectorized
  using 32 byte vectors
src/plain.cpp:33:26: optimized: loop vectorized
  using 16 byte vectors
```

The compiler generates both 256-bit (ymm) and 128-bit (xmm) versions; at runtime, the most efficient variant is selected. The key enabler is that the hash uses `mul32`, which maps to `vpmulld`—a native instruction. With a mul64 hash, GCC would only emit 128-bit code using the decomposed `vpmuludq`.

Vectorizability conditions: all are satisfied—fixed trip count, no inter-iteration dependencies, no function calls, `__restrict__` for aliasing, unit stride access, and a SIMD-native operation.

3 AVX2 Intrinsics Strategy

The intrinsics version processes **8 keys per iteration** using two ymm loads (4+4 keys) that are XOR-folded, packed into a single ymm of 8×uint32, multiplied with one `vpmulld`, shifted, and stored:

```
for (size_t i = 0; i < simd_end; i += 8) {
  __m256i vk0 = _mm256_load_si256(&keys[i]);
  __m256i vk1 = _mm256_load_si256(&keys[i+4]);
  // XOR fold: k_lo ^ k_hi
  __m256i x0 = _mm256_xor_si256(vk0,
               _mm256_srli_epi64(vk0, 32));
  __m256i x1 = _mm256_xor_si256(vk1,
               _mm256_srli_epi64(vk1, 32));
  // pack 4+4 -> 8 x uint32
  __m256i mixed = _mm256_permute2x128_si256(
    _mm256_permutevar8x32_epi32(x0, shuf),
    _mm256_permutevar8x32_epi32(x1, shuf), 0x20);
  // native mul32 + shift
  __m256i h = _mm256_srli_epi32(
    _mm256_mullo_epi32(mixed, va32), shift);
  _mm256_storeu_si256(&part_ids[i], h);
}
```

Instruction count: ∼12 SIMD instructions for 8 keys (1.5/key), vs. ∼3 scalar instructions per key. The theoretical speedup from instruction reduction alone would be $3/1.5 = 2\times$, but the kernel is memory-bound (see Section 4).

Correctness: the AVX2 binary verifies checksum identity against a scalar reference before benchmarking. Element-wise comparison is available for $N \leq 32$.

4 Performance Results and Analysis

All benchmarks run on **node09** (AMD EPYC 7301, Zen 1, 2.7 GHz boost, DDR4-2666) with exclusive SLURM allocation (`-partition=gpu-excl`), 11 repetitions, median reported.

Version	Throughput (Mkeys/s)	Speedup (vs baseline)	BW (GB/s)
Baseline (no-vec)	918	1.00×	11.0
Auto-vectorized	1310	1.43×	15.7
AVX2 intrinsics	1200	1.31×	14.4
CUDA kernel-only	67 320	73.4×	808
CUDA end-to-end	1 031	1.12×	12.4

Table 1: Performance at $N = 100$M, $P = 256$. BW computed as throughput × 12 B/key (8 B read + 4 B write).

4.1 Summary

4.2 Roofline Analysis: Why Only 1.3–1.4×

The kernel transfers 12 bytes per key (8 B input + 4 B output) and performs ∼4 operations (XOR, mul32, shift, store). The **operational intensity** is:

$$I = \frac{4 \text{ ops}}{12 \text{ bytes}} = 0.33 \text{ ops/byte} \tag{2}$$

Using the roofline model: $P = \min(R_{\text{peak}}, I \times B_{\text{peak}})$, with hardware parameters measured on node09:

- $B_{\text{peak}} = 16.4$ GB/s (single-core, measured via AVX2 copy)
- $R_{\text{peak}}^{\text{scalar}} = 5.17$ Gops/s
- $R_{\text{peak}}^{\text{AVX2}} = 15.28$ Gops/s

The attainable performance ceiling at $I = 0.33$ is $I \times B_{\text{peak}} = 5.47$ Gops/s. The auto-vectorized version achieves 5.24 Gops/s = **96% of the bandwidth ceiling**. Figure 1 confirms all three implementations lie on the bandwidth ramp.

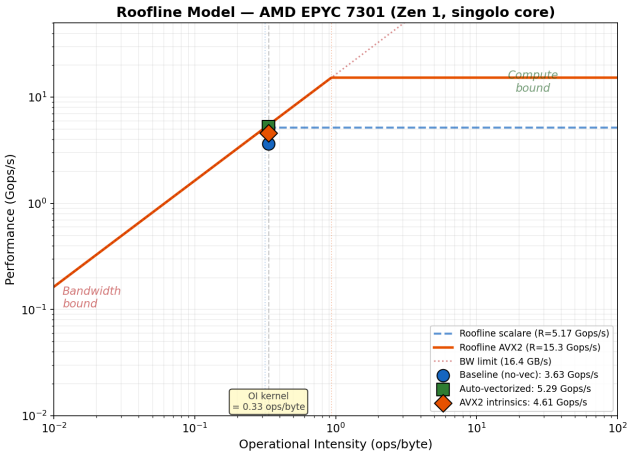

Figure 1: Roofline model. All kernel variants fall in the bandwidth-bound region ($I = 0.33$ ops/byte).

Why the speedup is only ∼1.3× instead of 8×: AVX2 theoretically processes 8 uint32 elements per instruction, but this only accelerates computation time (t_{comp}). Since the kernel is memory-bound ($t_{\text{mem}} > t_{\text{comp}}$), the speedup comes solely from more efficient bandwidth utilization—fewer total instructions means less frontend pressure and better prefetching.

Why autovec > AVX2 intrinsics: GCC applies loop unrolling and instruction scheduling that our intrinsics code does not benefit from, achieving higher effective bandwidth (15.7 vs 14.4 GB/s).

4.3 SIMD Efficiency and Amdahl's Interpretation

Applying standard parallel performance metrics to SIMD parallelism, where $p = 8$ (AVX2 256-bit, 8 uint32 lanes):

Metric	Autovec	AVX2
Speedup $S(p)$	1.46×	1.27×
Efficiency $E(p) = S/p$	18.2%	15.9%
Cost $C(p) = T_{\text{par}} \times p$	$5.5 \times T_{\text{seq}}$	$6.3 \times T_{\text{seq}}$

Table 2: SIMD metrics at $N = 200$M, $P = 256$, $p = 8$.

The efficiency of ∼16–18% is characteristic of memory-bound kernels: SIMD lanes spend most of their time stalled waiting for data. Interpreting via Amdahl's law, the effective serial fraction is $f \approx 64\%$, which corresponds to the memory transfer time that cannot be parallelized by SIMD. Even with $p \to \infty$ (wider SIMD or AVX-512), the theoretical maximum speedup would be only $1/f \approx 1.56\times$.

4.4 Parameter Sweeps

N-sweep (Figure 2): speedup is stable across all input sizes, from 1M to 200M keys. This confirms that the kernel is bandwidth-bound regardless of N: as N grows, both the baseline and vectorized versions scale linearly with data size, preserving a constant speedup ratio.

P-sweep: throughput and speedup are independent of P (all curves flat across $P \in [2, 1024]$), confirming that changing P only modifies the shift constant—memory traffic and compute per key are unchanged.

Key-space sweep: throughput is invariant to the key-space size (duplicate rate), as expected for a branchless hash function with constant cost per key.

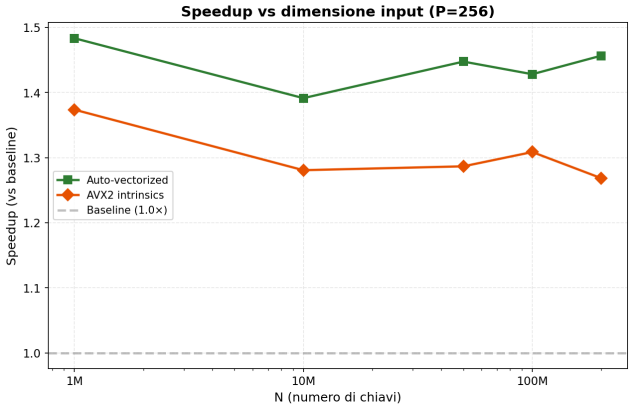

Figure 2: Speedup vs. input size N ($P = 256$). Both auto-vectorized and AVX2 intrinsics maintain stable speedup across all N, consistent with a bandwidth-bound kernel.

4.5 Measurement Stability

Each configuration was executed 11 times on an exclusively allocated node (`-exclusive` SLURM flag, no co-tenants). We report the median and compute the coefficient of variation $\mathrm{CoV} = \sigma/\tilde{x}$ as a stability indicator.

Impl.	Mean CoV	Max CoV	Measures
Baseline	1.98%	4.64%	20
Autovec	0.39%	3.27%	15
AVX2	3.46%	4.41%	20

Table 3: CoV statistics across all 55 measurements. None exceeds 5%.

The autovec version is the most stable (mean CoV = 0.39%) because GCC's optimized scheduling produces highly deterministic memory access patterns. The AVX2 intrinsics version shows higher variance (mean 3.46%), likely due to the rigid instruction sequence interacting less predictably with the memory subsystem. Baseline CoV is higher at small N (4.64% at $N = 1\mathrm{M}$) where timer granularity and startup overhead are relatively significant, and drops below 0.1% for $N \geq 50\mathrm{M}$ (Figure 3).

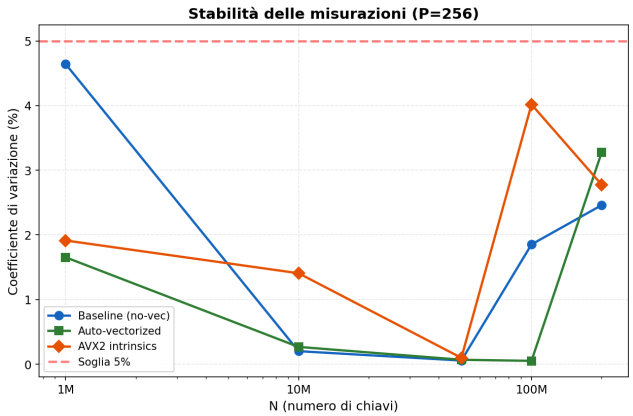

Figure 3: Coefficient of variation vs. N. All points below the 5% threshold (dashed red). CoV generally decreases with N as the measurement duration grows relative to system noise.

5 CUDA Implementation

The CUDA kernel applies the same XOR-fold + mul32 hash, one thread per key, with 256 threads/block. Correctness is verified by checksum comparison against the CPU reference.

The kernel alone achieves 67 320 Mkeys/s = 808 GB/s, utilizing **81%** of the A30's HBM2 peak (993 GB/s). However, PCIe 4.0 transfers dominate end-to-end time (**98.5%**), making GPU offload of this isolated kernel no faster than the CPU. GPU offload would become advantageous only if the data were already resident in GPU memory or if subsequent pipeline stages also executed on GPU.

Phase	Time (ms)	BW (GB/s)	% total
H→D transfer	63.2	12.7	65.2%
Kernel	1.5	808	1.5%
D→H transfer	32.3	12.4	33.3%
Total (end-to-end)	**97.0**	—	100%

Table 4: CUDA timing breakdown at $N = 100\mathrm{M}$, $P = 256$.

6 Conclusions

The partition mapping kernel is a textbook memory-bound workload. At $I = 0.33$ ops/byte, performance is capped by DRAM bandwidth on both CPU and GPU (via PCIe for end-to-end). SIMD (both auto-vectorized and intrinsics) provides a modest 1.3–1.4× speedup by improving bandwidth utilization efficiency, not by reducing computation time.

The critical design insight was choosing a 32-bit hash function (`vpmulld`-native) over a 64-bit one (requiring 3× `vpmuludq` decomposition). With the 64-bit hash, AVX2 was actually *slower* than scalar code—the decomposition overhead exceeded the parallelism benefit in a bandwidth-limited regime.

Further speedup would require multi-core parallelism to aggregate bandwidth across memory channels (Module 2), or algorithmic changes to increase operational intensity (e.g., fusing the mapping with downstream join operations).